

CONTENTS

PREFACE	xi
PART I: THEORY	1
1 INTRODUCTION TO THE THEORY OF COMPUTATION	3
1.1 Mathematical Preliminaries and Notation	5
Sets	5
Functions and Relations	8
Graphs and Trees	10
Proof Techniques	11
1.2 Three Basic Concepts	19
Languages	19
Grammars	22
Automata	28
1.3 Some Applications*	32
2 FINITE AUTOMATA	39
2.1 Deterministic Finite Accepters	40
Deterministic Accepters and Transition Graphs	41
Languages and Dfa's	43
Regular Languages	48
2.2 Nondeterministic Finite Accepters	53
Definition of a Nondeterministic Acceptor	54
Why Nondeterminism?	58

2.3	Equivalence of Deterministic and Nondeterministic Finite Accepters	61
2.4	Reduction of the Number of States in Finite Automata* . . .	69
3	REGULAR LANGUAGES AND REGULAR GRAMMARS	77
3.1	Regular Expressions	78
	Formal Definition of a Regular Expression	78
	Languages Associated with Regular Expressions	79
3.2	Connection Between Regular Expressions and Regular Languages	84
	Regular Expressions Denote Regular Languages	84
	Regular Expressions for Regular Languages	87
	Regular Expressions for Describing Simple Patterns	93
3.3	Regular Grammars	97
	Right- and Left-Linear Grammars	97
	Right-Linear Grammars Generate Regular Languages	99
	Right-Linear Grammars for Regular Languages	101
	Equivalence of Regular Languages and Regular Grammars . .	103
4	PROPERTIES OF REGULAR LANGUAGES	107
4.1	Closure Properties of Regular Languages	108
	Closure under Simple Set Operations	108
	Closure under Other Operations	111
4.2	Elementary Questions about Regular Languages	120
4.3	Identifying Nonregular Languages	123
	Using the Pigeonhole Principle	124
	A Pumping Lemma	125
5	CONTEXT-FREE LANGUAGES	137
5.1	Context-Free Grammars	138
	Examples of Context-Free Languages	139
	Leftmost and Rightmost Derivations	141
	Derivation Trees	142
	Relation Between Sentential Forms and Derivation Trees . .	145
5.2	Parsing and Ambiguity	149
	Parsing and Membership	149
	Ambiguity in Grammars and Languages	154
5.3	Context-Free Grammars and Programming Languages	160
6	SIMPLIFICATION OF CONTEXT-FREE GRAMMARS AND NORMAL FORMS	165
6.1	Methods for Transforming Grammars	166
	A Useful Substitution Rule	167
	Removing Useless Productions	169

	Removing λ -Productions	173
	Removing Unit-Productions	175
6.2	Two Important Normal Forms	182
	Chomsky Normal Form	182
	Greibach Normal Form	185
6.3	A Membership Algorithm for Context-Free Grammars*	189
7	PUSHDOWN AUTOMATA	193
7.1	Nondeterministic Pushdown Automata	194
	Definition of a Pushdown Automaton	195
	The Language Accepted by a Pushdown Automaton	198
7.2	Pushdown Automata and Context-Free Languages	204
	Pushdown Automata for Context-Free Languages	204
	Context-Free Grammars for Pushdown Automata	209
7.3	Deterministic Pushdown Automata and Deterministic Context-Free Languages	216
7.4	Grammars for Deterministic Context-Free Languages*	221
8	PROPERTIES OF CONTEXT-FREE LANGUAGES	227
8.1	Two Pumping Lemmas	228
	A Pumping Lemma for Context-Free Languages	228
	A Pumping Lemma for Linear Languages	233
8.2	Closure Properties and Decision Algorithms for Context-Free Languages	237
	Closure of Context-Free Languages	238
	Some Decidable Properties of Context-Free Languages	242
9	TURING MACHINES	247
9.1	The Standard Turing Machine	248
	Definition of a Turing Machine	248
	Turing Machines as Language Accepters	255
	Turing Machines as Transducers	258
9.2	Combining Turing Machines for Complicated Tasks	266
9.3	Turing's Thesis	271
10	OTHER MODELS OF TURING MACHINES	275
10.1	Minor Variations on the Turing Machine Theme	276
	Equivalence of Classes of Automata	276
	Turing Machines with a Stay-Option	277
	Turing Machines with Semi-Infinite Tape	279
	The Off-Line Turing Machine	281
10.2	Turing Machines with More Complex Storage	284
	Multitape Turing Machines	284
	Multidimensional Turing Machines	287
10.3	Nondeterministic Turing Machines	289

10.4	A Universal Turing Machine	292
10.5	Linear Bounded Automata	297
11	A HIERARCHY OF FORMAL LANGUAGES AND AUTOMATA	301
11.1	Recursive and Recursively Enumerable Languages	302
	Languages That Are Not Recursively Enumerable	304
	A Language That Is Not Recursively Enumerable	305
	A Language That Is Recursively Enumerable but Not Recursive	307
11.2	Unrestricted Grammars	309
11.3	Context-Sensitive Grammars and Languages	315
	Context-Sensitive Languages and Linear Bounded Automata	316
	Relation Between Recursive and Context-Sensitive Languages	318
11.4	The Chomsky Hierarchy	321
12	LIMITS OF ALGORITHMIC COMPUTATION	325
12.1	Some Problems That Cannot Be Solved by Turing Machines .	326
	Computability and Decidability	326
	The Turing Machine Halting Problem	327
	Reducing One Undecidable Problem to Another	331
12.2	Undecidable Problems for Recursively Enumerable Languages	335
12.3	The Post Correspondence Problem	339
12.4	Undecidable Problems for Context-Free Languages	345
12.5	A Question of Efficiency	348
13	OTHER MODELS OF COMPUTATION	351
13.1	Recursive Functions	353
	Primitive Recursive Functions	354
	Ackermann's Function	358
	μ Recursive Functions	359
13.2	Post Systems	362
13.3	Rewriting Systems	366
	Matrix Grammars	366
	Markov Algorithms	367
	L-Systems	369
14	AN OVERVIEW OF COMPUTATIONAL COMPLEXITY	371
14.1	Efficiency of Computation	372
14.2	Turing Machine Models and Complexity	374
14.3	Language Families and Complexity Classes	378

14.4	The Complexity Classes P and NP	381
14.5	Some NP Problems	383
14.6	Polynomial-Time Reduction	386
14.7	NP-Completeness and an Open Question	389
PART II: APPLICATIONS		393
15	COMPILERS AND PARSING	395
15.1	Compilers	396
15.2	Top-Down vs. Bottom-Up Parsing	404
15.3	FIRST Function	409
15.4	FOLLOW Function	416
16	LL PARSING	423
16.1	Context-Free Grammar Conversion to Nondeterministic Pushdown Automaton	424
	Pushdown Automata for <i>LL</i> Parsing	424
	Algorithm to Convert Context-Free Grammar to NPDA for <i>LL</i> Parsing	424
16.2	<i>LL</i> (1) Parse Table	429
	<i>LL</i> (1) Parse Table	429
16.3	<i>LL</i> (1) Parsing Algorithm	431
	<i>LL</i> (1) Parsing Algorithm	432
16.4	<i>LL</i> (<i>k</i>) Parsing	435
17	LR PARSING	441
17.1	Context-Free Grammar Conversion to NPDA	443
	Algorithm to Convert Context-Free Grammar to NPDA for <i>LR</i> Parsing	443
17.2	Items and Closure	449
17.3	DFA Models the <i>LR</i> Parsing Stack	453
	Algorithm to Build DFA Modeling LR Parsing	454
17.4	<i>LR</i> (1) Parse Table	467
	<i>LR</i> (1) Parsing Actions	468
	<i>LR</i> (1) Parse Table Algorithm	468
17.5	<i>LR</i> (1) Parsing Algorithm	475
	<i>LR</i> (1) Parsing Algorithm	476
17.6	<i>LR</i> (1) Parsing with λ -Productions	481
17.7	<i>LR</i> (1) Parsing Conflicts	492
APPENDIX A FINITE-STATE TRANSDUCERS		499
A.1	A General Framework	499
A.2	Mealy Machines	500
A.3	Moore Machines	502
A.4	Moore and Mealy Machine Equivalence	504

A.5	Mealy Machine Minimization	508
A.6	Moore Machine Minimization	513
A.7	Limitations of Finite-State Transducers	514
APPENDIX B JFLAP: A USEFUL TOOL		517
ANSWERS: SOLUTIONS AND HINTS FOR SELECTED EXERCISES		519
REFERENCES FOR FURTHER READING		573
INDEX		575

PREFACE

This book is designed for an introductory course on formal languages, automata, computability, and related matters. These topics form a major part of what is known as the theory of computation. A course on this subject matter is now standard in the computer science curriculum and is often taught fairly early in the program. Hence, the prospective audience for this book consists primarily of sophomores and juniors majoring in computer science or computer engineering.

Prerequisites for the material in this book are a knowledge of some higher-level programming language (commonly C, C++, PythonTM, or JavaTM) and familiarity with the fundamentals of data structures and algorithms. A course in discrete mathematics that includes set theory, functions, relations, logic, and elements of mathematical reasoning is essential. Such a course is part of the standard introductory computer science curriculum.

The study of the theory of computation has several purposes, most importantly (1) to familiarize students with the foundations and principles of computer science, (2) to teach material that is useful in subsequent courses, and (3) to strengthen students' ability to carry out formal and rigorous mathematical arguments. The presentation I have chosen for this text favors the first two purposes, although I would argue that it also serves the third. To present ideas clearly and to give students insight into the material, the text stresses intuitive motivation and illustration of ideas through examples. When there is a choice, I prefer arguments that are easily grasped to those that are concise and elegant but difficult in concept. I state definitions and theorems precisely and give the motivation for proofs

but often leave out the routine and tedious details. I believe that this is desirable for pedagogical reasons. Many proofs are unexciting applications of induction or contradiction with differences that are specific to particular problems. Presenting such arguments in full detail is not only unnecessary, but it interferes with the flow of the story. Therefore, quite a few of the proofs are brief, and someone who insists on completeness may consider them lacking in detail. I do not see this as a drawback. Mathematical skills are not the by-product of reading someone else's arguments, but they come from thinking about the essence of a problem, discovering ideas suitable to make the point, then carrying them out in precise detail. The latter skill certainly has to be learned, and I think that the proof sketches in this text provide very appropriate starting points for such a practice.

Computer science students sometimes view a course in the theory of computation as unnecessarily abstract and of no practical consequence. To convince them otherwise, one needs to appeal to their specific interests and strengths, such as tenacity and inventiveness in dealing with hard-to-solve problems. Because of this, my approach emphasizes learning through problem solving.

By a problem-solving approach, I mean that students learn the material primarily through problem-type illustrative examples that show the motivation behind the concepts, as well as their connection to the theorems and definitions. At the same time, the examples may involve a nontrivial aspect, for which students must discover a solution. In such an approach, homework exercises contribute to a major part of the learning process. The exercises at the end of each section are designed to illuminate and illustrate the material and call on students' problem-solving ability at various levels. Some of the exercises are fairly simple, picking up where the discussion in the text leaves off and asking students to carry on for another step or two. Other exercises are very difficult, challenging even the best minds. A good mix of such exercises can be a very effective teaching tool. Students need not be asked to solve all problems, but should be assigned those that support the goals of the course and the viewpoint of the instructor. Computer science curricula differ from institution to institution; while a few emphasize the theoretical side, others are almost entirely oriented toward practical application. I believe that this text can serve either of these extremes, provided that the exercises are selected carefully with the students' background and interests in mind. At the same time, the instructor needs to inform the students about the level of abstraction that is expected of them. This is particularly true of the proof-oriented exercises. When I say "prove that" or "show that," I have in mind that the student should think about how a proof can be constructed and then produce a clear argument. How formal such a proof should be needs to be determined by the instructor, and students should be given guidelines on this early in the course.

The content of the text is appropriate for a one-semester course. Most of the material can be covered, although some choice of emphasis will have

to be made. In my classes, I generally gloss over proofs, giving just enough coverage to make the result plausible, and then ask students to read the rest on their own. Overall, though, little can be skipped entirely without potential difficulties later on. A few sections, which are marked with an asterisk, can be omitted without loss to later material. Most of the material, however, is essential and must be covered.

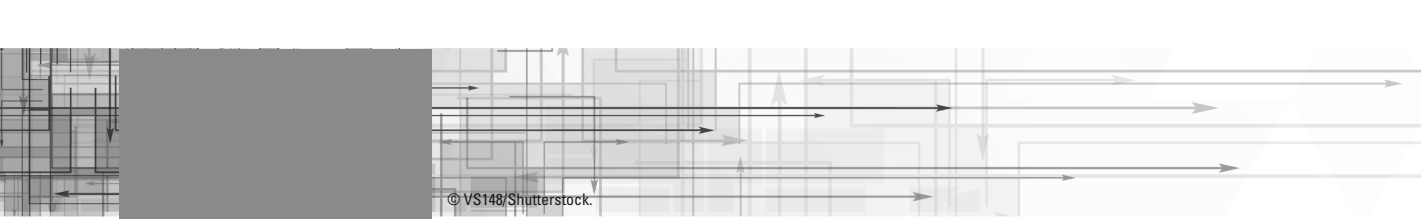
Appendix B briefly introduces JFLAP, an interactive software tool available for free at www.jflap.org that is of great help in both learning and teaching the material in this book. It aids in understanding the concepts and is a great time saver in the actual construction of the solutions to the exercises. I highly recommend incorporating JFLAP into the course.

The seventh edition of this book features two additions. The first is a large number of new exercises, collected at the chapter ends, under the heading Introductory Exercises to distinguish them from the already existing Exercises. These new exercises are largely very simple and require only an understanding of the concepts. They are intended as a bridge to the often much more difficult Exercises. The instructor can decide where and for whom these new exercises can be of help. Chapters 1–14 of the sixth edition, with the new exercises, are now reorganized as Part I: Theory.

Substantial new material comes in Part II: Applications, where we discuss some of the important issues in compiler construction in the context of the material in Part I. This new material comes in response to perennial student questions such as “How does this theory apply to the mostly practical matters of computer science?” or even “Why do we have to know all this abstract stuff?” We hope we have given a satisfactory response to such questions.

How can this material be integrated into an existing course? Instructors who cannot find the time for it might use Part II as suggested reading for curious students. But the material is important and attractive to many students, so we recommend that some attempt to cover it in class be made. Part I introduces the main aspects of the theory of computation, but not everything there is essential. Not only the starred sections, but many sections in the latter part can be treated lightly or skipped altogether without loss of continuity. Doing so might free a week or so for Part II. This coverage can benefit many students. Those who never take a course on compilers will get some understanding of what is involved in compiler design; for others who do, it provides a first look at what will be covered later in more detail.

Peter Linz
Susan H. Rodger

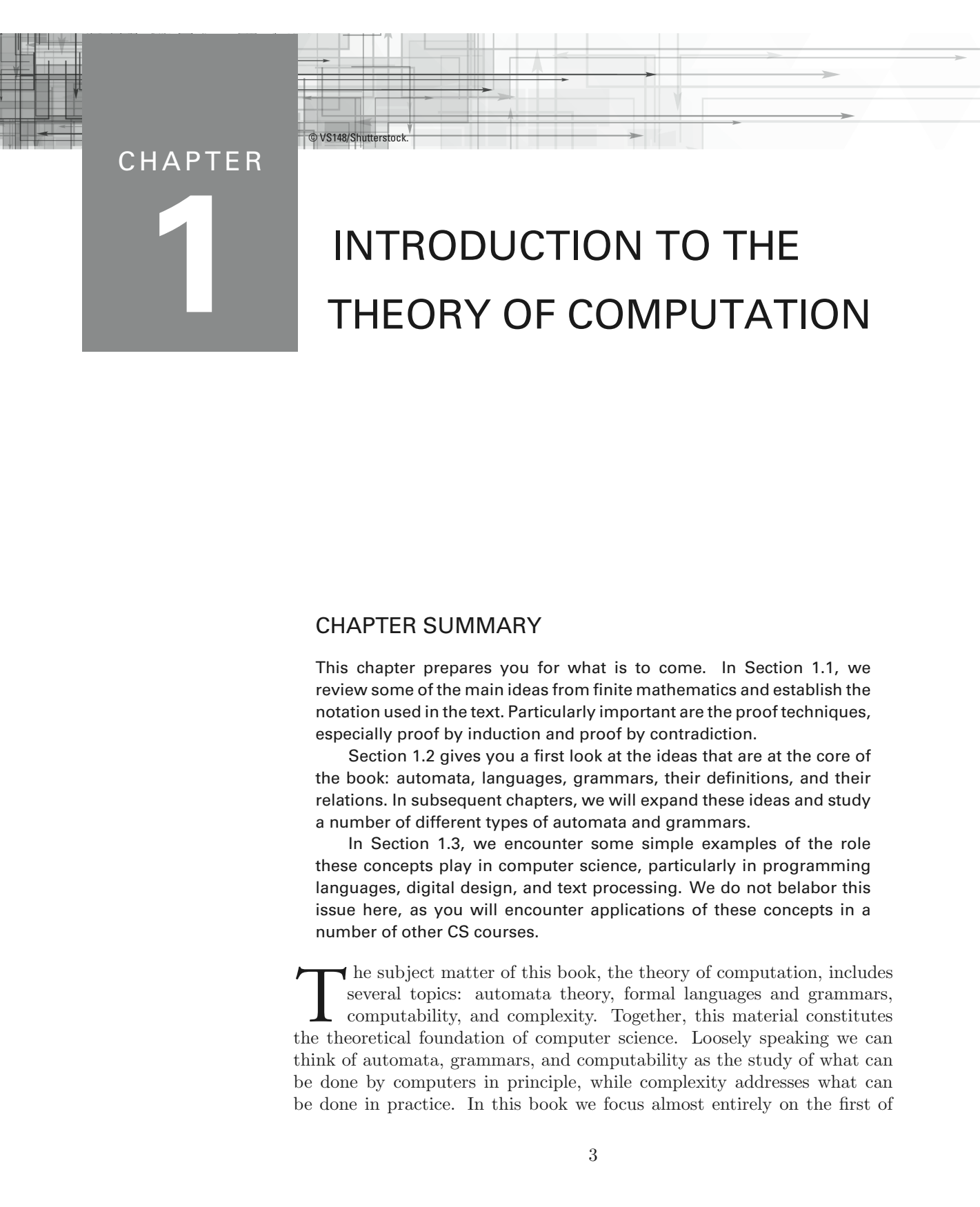


PART

I

© VS148/Shutterstock.

THEORY



CHAPTER

1

INTRODUCTION TO THE THEORY OF COMPUTATION

CHAPTER SUMMARY

This chapter prepares you for what is to come. In Section 1.1, we review some of the main ideas from finite mathematics and establish the notation used in the text. Particularly important are the proof techniques, especially proof by induction and proof by contradiction.

Section 1.2 gives you a first look at the ideas that are at the core of the book: automata, languages, grammars, their definitions, and their relations. In subsequent chapters, we will expand these ideas and study a number of different types of automata and grammars.

In Section 1.3, we encounter some simple examples of the role these concepts play in computer science, particularly in programming languages, digital design, and text processing. We do not belabor this issue here, as you will encounter applications of these concepts in a number of other CS courses.

The subject matter of this book, the theory of computation, includes several topics: automata theory, formal languages and grammars, computability, and complexity. Together, this material constitutes the theoretical foundation of computer science. Loosely speaking we can think of automata, grammars, and computability as the study of what can be done by computers in principle, while complexity addresses what can be done in practice. In this book we focus almost entirely on the first of

these concerns. We will study various automata, see how they are related to languages and grammars, and investigate what can and cannot be done by digital computers. Although this theory has many uses, it is inherently abstract and mathematical.

Computer science is a practical discipline. Those who work in it often have a marked preference for useful and tangible problems over theoretical speculation. This is certainly true of computer science students who are concerned mainly with difficult applications from the real world. Theoretical questions interest them only if they help in finding good solutions. This attitude is appropriate since without applications there would be little interest in computers. But given this practical orientation, one might well ask, “Why study theory?”

The first answer is that theory provides concepts and principles that help us understand the general nature of the discipline. The field of computer science includes a wide range of special topics, from machine design to programming. The use of computers in the real world involves a wealth of specific detail that must be learned for a successful application. This makes computer science a very diverse and broad discipline. But in spite of this diversity, there are some common underlying principles. To study these basic principles, we construct abstract models of computers and computation. These models embody the important features that are common to both hardware and software and that are essential to many of the special and complex constructs we encounter while working with computers. Even when such models are too simple to be applicable immediately to real-world situations, the insights we gain from studying them provide the foundation on which specific development is based. This approach is, of course, not unique to computer science. The construction of models is one of the essentials of any scientific discipline, and the usefulness of a discipline is often dependent on the existence of simple, yet powerful, theories and laws.

A second, and perhaps not so obvious, answer is that the ideas we will discuss have some immediate and important applications. The fields of digital design, programming languages, and compilers are the most obvious examples, but there are many others. The concepts we study here run like a thread through much of computer science, from operating systems to pattern recognition.

The third answer is one of which we hope to convince the reader. The subject matter is intellectually stimulating and fun. It provides many challenging, puzzle-like problems that can lead to some sleepless nights. This is problem solving in its pure essence.

In this book, we will look at models that represent features at the core of all computers and their applications. To model the hardware of a computer, we introduce the notion of an **automaton** (plural, **automata**). An automaton is a construct that possesses all the indispensable features of a digital computer. It accepts input, produces output, may have some temporary storage, and can make decisions in transforming the input into

the output. A **formal language** is an abstraction of the general characteristics of programming languages. A formal language consists of a set of symbols and some rules of formation by which these symbols can be combined into entities called sentences. A formal language is the set of all sentences permitted by the rules of formation. Although some of the formal languages we study here are simpler than programming languages, they have many of the same essential features. We can learn a great deal about programming languages from formal languages. Finally, we will formalize the concept of a mechanical computation by giving a precise definition of the term **algorithm** and study the kinds of problems that are (and are not) suitable for solution by such mechanical means. In the course of our study, we will show the close connection between these abstractions and investigate the conclusions we can derive from them.

In the first chapter, we look at these basic ideas in a very broad way to set the stage for later work. In Section 1.1, we review the main ideas from mathematics that will be required. While intuition will frequently be our guide in exploring ideas, the conclusions we draw will be based on rigorous arguments. This will involve some mathematical machinery, although the requirements are not extensive. The reader will need a reasonably good grasp of the terminology and of the elementary results of set theory, functions, and relations. Trees and graph structures will be used frequently, although little is needed beyond the definition of a labeled, directed graph. Perhaps the most stringent requirement is the ability to follow proofs and an understanding of what constitutes proper mathematical reasoning. This includes familiarity with the basic proof techniques of deduction, induction, and proof by contradiction. We will assume that the reader has this necessary background. Section 1.1 is included to review some of the main results that will be used and to establish a notational common ground for subsequent discussion.

In Section 1.2, we take a first look at the central concepts of languages, grammars, and automata. These concepts occur in many specific forms throughout the book. In Section 1.3, we give some simple applications of these general ideas to illustrate that these concepts have widespread uses in computer science. The discussion in these two sections will be intuitive rather than rigorous. Later, we will make all of this much more precise, but for the moment, the goal is to get a clear picture of the concepts with which we are dealing.

1.1 MATHEMATICAL PRELIMINARIES AND NOTATION

Sets

A **set** is a collection of elements, without any structure other than membership. To indicate that x is an element of the set S , we write $x \in S$.

The statement that x is not in S is written $x \notin S$. A set can be specified by enclosing some description of its elements in curly braces; for example, the set of integers 0, 1, 2 is shown as

$$S = \{0, 1, 2\}.$$

Ellipses are used whenever the meaning is clear. Thus, $\{a, b, \dots, z\}$ stands for all the lowercase letters of the English alphabet, while $\{2, 4, 6, \dots\}$ denotes the set of all positive even integers. When the need arises, we use more explicit notation, in which we write

$$S = \{i : i > 0, i \text{ is even}\} \quad (1.1)$$

for the last example. We read this as “ S is the set of all i , such that i is greater than zero, and i is even,” implying, of course, that i is an integer.

The usual set operations are **union** ($\cup$), **intersection** ($\cap$), and **difference** ($-$) defined as

$$\begin{aligned} S_1 \cup S_2 &= \{x : x \in S_1 \text{ or } x \in S_2\}, \\ S_1 \cap S_2 &= \{x : x \in S_1 \text{ and } x \in S_2\}, \\ S_1 - S_2 &= \{x : x \in S_1 \text{ and } x \notin S_2\}. \end{aligned}$$

Another basic operation is **complementation**. The complement of a set S , denoted by $\overline{S}$, consists of all elements not in S . To make this meaningful, we need to know what the **universal set** U of all possible elements is. If U is specified, then

$$\overline{S} = \{x : x \in U, x \notin S\}.$$

The set with no elements, called the **empty set** or the **null set**, is denoted by $\emptyset$. From the definition of a set, it is obvious that

$$\begin{aligned} S \cup \emptyset &= S - \emptyset = S, \\ S \cap \emptyset &= \emptyset, \\ \overline{\emptyset} &= U, \\ \overline{\overline{S}} &= S. \end{aligned}$$

The following useful identities, known as **DeMorgan's laws**,

$$\overline{S_1 \cup S_2} = \overline{S_1} \cap \overline{S_2}, \quad (1.2)$$

$$\overline{S_1 \cap S_2} = \overline{S_1} \cup \overline{S_2}, \quad (1.3)$$

are needed on several occasions.

A set S_1 is said to be a **subset** of S if every element of S_1 is also an element of S . We write this as

$$S_1 \subseteq S.$$

If $S_1 \subseteq S$, but S contains an element not in S_1 , we say that S_1 is a **proper subset** of S ; we write this as

$$S_1 \subset S.$$

If S_1 and S_2 have no common element, that is, $S_1 \cap S_2 = \emptyset$, then the sets are said to be **disjoint**.

A set is said to be **finite** if it contains a finite number of elements; otherwise it is **infinite**. The size of a finite set is the number of elements in it; this is denoted by $|S|$.

A given set normally has many subsets. The set of all subsets of a set S is called the **powerset** of S and is denoted by 2^S . Observe that 2^S is a set of sets.

EXAMPLE 1.1

If S is the set $\{a, b, c\}$, then its powerset is

$$2^S = \{\emptyset, \{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}, \{a, b, c\}\}.$$

Here $|S| = 3$ and $|2^S| = 8$. This is an instance of a general result; if S is finite, then

$$|2^S| = 2^{|S|}.$$

In many of our examples, the elements of a set are ordered sequences of elements from other sets. Such sets are said to be the **Cartesian product** of other sets. For the Cartesian product of two sets, which itself is a set of ordered pairs, we write

$$S = S_1 \times S_2 = \{(x, y) : x \in S_1, y \in S_2\}.$$

EXAMPLE 1.2

Let $S_1 = \{2, 4\}$ and $S_2 = \{2, 3, 5, 6\}$. Then

$$S_1 \times S_2 = \{(2, 2), (2, 3), (2, 5), (2, 6), (4, 2), (4, 3), (4, 5), (4, 6)\}.$$

Note that the order in which the elements of a pair are written matters. The pair $(4, 2)$ is in $S_1 \times S_2$, but $(2, 4)$ is not.

The notation is extended in an obvious fashion to the Cartesian product of more than two sets; generally

$$S_1 \times S_2 \times \cdots \times S_n = \{(x_1, x_2, \dots, x_n) : x_i \in S_i\}.$$

A set can be divided by separating it into a number of subsets. Suppose that $S_1, S_2, \dots, S_n$ are subsets of a given set S and that the following holds:

1. The subsets $S_1, S_2, \dots, S_n$ are mutually disjoint;
2. $S_1 \cup S_2 \cup \dots \cup S_n = S$;
3. none of the S_i is empty.

Then $S_1, S_2, \dots, S_n$ is called a **partition** of S .

Functions and Relations

A **function** is a rule that assigns to elements of one set a unique element of another set. If f denotes a function, then the first set is called the **domain** of f , and the second set is its **range**. We write

$$f: S_1 \rightarrow S_2$$

to indicate that the domain of f is a subset of S_1 and that the range of f is a subset of S_2 . If the domain of f is all of S_1 , we say that f is a **total function** on S_1 ; otherwise f is said to be a **partial function**.

In many applications, the domain and range of the functions involved are in the set of positive integers. Furthermore, we are often interested only in the behavior of these functions as their arguments become very large. In such cases an understanding of the growth rates may suffice, and a common order of magnitude notation can be used. Let $f(n)$ and $g(n)$ be functions whose domain is a subset of the positive integers. If there exists a positive constant c such that for all sufficiently large n

$$f(n) \leq c|g(n)|,$$

we say that f has **order at most** g . We write this as

$$f(n) = O(g(n)).$$

If

$$|f(n)| \geq c|g(n)|,$$

then f has **order at least** g , for which we use

$$f(n) = \Omega(g(n)).$$

Finally, if there exist constants c_1 and c_2 such that

$$c_1|g(n)| \leq |f(n)| \leq c_2|g(n)|,$$

f and g have the **same order of magnitude**, expressed as

$$f(n) = \Theta(g(n)).$$

In this order-of-magnitude notation, we ignore multiplicative constants and lower-order terms that become negligible as n increases.

EXAMPLE 1.3

Let

$$\begin{aligned}f(n) &= 2n^2 + 3n, \\g(n) &= n^3, \\h(n) &= 10n^2 + 100.\end{aligned}$$

Then

$$\begin{aligned}f(n) &= O(g(n)), \\g(n) &= \Omega(h(n)), \\f(n) &= \Theta(h(n)).\end{aligned}$$

In order-of-magnitude notation, the symbol $=$ should not be interpreted as equality, and order-of-magnitude expressions cannot be treated like ordinary expressions. Manipulations such as

$$O(n) + O(n) = 2O(n)$$

are not sensible and can lead to incorrect conclusions. Still, if used properly, the order-of-magnitude arguments can be effective, as we will see in later chapters.

Some functions can be represented by a set of pairs

$$\{(x_1, y_1), (x_2, y_2), \dots\},$$

where x_i is an element in the domain of the function, and y_i is the corresponding value in its range. For such a set to define a function, each x_i can occur at most once as the first element of a pair. If this is not satisfied, the set is called a **relation**. Relations are more general than functions: In a function each element of the domain has exactly one associated element in the range; in a relation there may be several such elements in the range.

One kind of relation is that of **equivalence**, a generalization of the concept of equality (identity). To indicate that a pair (x, y) is in an equivalence relation, we write

$$x \equiv y.$$

A relation denoted by $\equiv$ is considered an equivalence if it satisfies three rules: the reflexivity rule

$$x \equiv x \text{ for all } x;$$

the symmetry rule

$$\text{if } x \equiv y, \text{ then } y \equiv x;$$

and the transitivity rule

$$\text{if } x \equiv y \text{ and } y \equiv z, \text{ then } x \equiv z.$$

EXAMPLE 1.4

On the set of nonnegative integers, we can define a relation

$$x \equiv y$$

if and only if

$$x \bmod 3 = y \bmod 3.$$

Then $2 \equiv 5$, $12 \equiv 0$, and $0 \equiv 36$. Clearly this is an equivalence relation, as it satisfies reflexivity, symmetry, and transitivity.

If S is a set on which we have a defined equivalence relation, then we can use this equivalence to partition the set into **equivalence classes**. Each equivalence class contains all and only equivalent elements.

Graphs and Trees

A graph is a construct consisting of two finite sets, the set $V = \{v_1, v_2, \dots, v_n\}$ of **vertices** and the set $E = \{e_1, e_2, \dots, e_m\}$ of **edges**. Each edge is a pair of vertices from V , for instance,

$$e_i = (v_j, v_k)$$

is an edge from v_j to v_k . We say that the edge e_i is an outgoing edge for v_j and an incoming edge for v_k . Such a construct is actually a directed graph (digraph), since we associate a direction (from v_j to v_k) with each edge. Graphs may be labeled, a label being a name or other information associated with parts of the graph. Both vertices and edges may be labeled.

Graphs are conveniently visualized by diagrams in which the vertices are represented as circles and the edges as lines with arrows connecting the vertices. The graph with vertices $\{v_1, v_2, v_3\}$ and edges $\{(v_1, v_3), (v_3, v_1), (v_3, v_2), (v_3, v_3)\}$ is depicted in Figure 1.1.

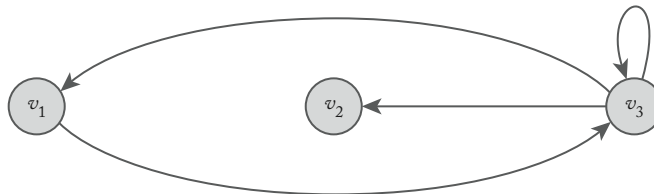


FIGURE 1.1

A sequence of edges $(v_i, v_j), (v_j, v_k), \dots, (v_m, v_n)$ is said to be a **walk** from v_i to v_n . The length of a walk is the total number of edges traversed in going from the initial vertex to the final one. A walk in which no edge is repeated is said to be a **path**; a path is **simple** if no vertex is repeated. A walk from v_i to itself with no repeated edges is called a **cycle** with **base** v_i . If no vertices other than the base are repeated in a cycle, then it is said to be simple. In Figure 1.1, $(v_1, v_3), (v_3, v_2)$ is a simple path from v_1 to v_2 . The sequence of edges $(v_1, v_3), (v_3, v_3), (v_3, v_1)$ is a cycle, but not a simple one. If the edges of a graph are labeled, we can talk about the label of a walk. This label is the sequence of edge labels encountered when the path is traversed. Finally, an edge from a vertex to itself is called a **loop**. In Figure 1.1, there is a loop on vertex v_3 .

On several occasions, we will refer to an algorithm for finding all simple paths between two given vertices (or all simple cycles based on a vertex). If we do not concern ourselves with efficiency, we can use the following obvious method. Starting from the given vertex, say v_i , list all outgoing edges $(v_i, v_k), (v_i, v_l), \dots$. At this point, we have all paths of length one starting at v_i . For all vertices $v_k, v_l, \dots$ so reached, we list all outgoing edges as long as they do not lead to any vertex already used in the path we are constructing. After we do this, we will have all simple paths of length two originating at v_i . We continue this until all possibilities are accounted for. Since there are only a finite number of vertices, we will eventually list all simple paths beginning at v_i . From these we select those ending at the desired vertex.

Trees are a particular type of graph. A tree is a directed graph that has no cycles and that has one distinct vertex, called the **root**, such that there is exactly one path from the root to every other vertex. This definition implies that the root has no incoming edges and that there are some vertices without outgoing edges. These are called the **leaves** of the tree. If there is an edge from v_i to v_j , then v_i is said to be the **parent** of v_j , and v_j the **child** of v_i . The **level** associated with each vertex is the number of edges in the path from the root to the vertex. The **height** of the tree is the largest level number of any vertex. These terms are illustrated in Figure 1.2.

At times, we want to associate an ordering with the nodes at each level; in such cases we talk about **ordered trees**.

More details on graphs and trees can be found in most books on discrete mathematics.

Proof Techniques

An important requirement for reading this text is the ability to follow proofs. In mathematical arguments, we employ the accepted rules of deductive reasoning, and many proofs are simply a sequence of such steps. Two special proof techniques are used so frequently that it is appropriate to review them briefly. These are **proof by induction** and **proof by contradiction**.

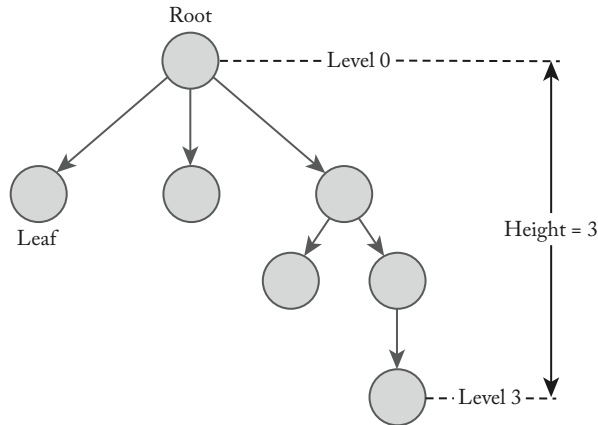


FIGURE 1.2

Induction is a technique by which the truth of a number of statements can be inferred from the truth of a few specific instances. Suppose we have a sequence of statements $P_1, P_2, \dots$ we want to prove to be true. Furthermore, suppose also that the following holds:

1. For some $k \geq 1$, we know that $P_1, P_2, \dots, P_k$ are true.
2. The problem is such that for any $n \geq k$, the truths of $P_1, P_2, \dots, P_n$ imply the truth of P_{n+1} .

We can then use induction to show that every statement in this sequence is true.

In a proof by induction, we argue as follows: From Condition 1 we know that the first k statements are true. Then Condition 2 tells us that P_{k+1} also must be true. But now that we know that the first $k+1$ statements are true, we can apply Condition 2 again to claim that P_{k+2} must be true, and so on. We need not explicitly continue this argument, because the pattern is clear. The chain of reasoning can be extended to any statement. Therefore, every statement is true.

The starting statements $P_1, P_2, \dots, P_k$ are called the **basis** of the induction. The step connecting P_n with P_{n+1} is called the **inductive step**. The inductive step is generally made easier by the **inductive assumption** that $P_1, P_2, \dots, P_n$ are true, then argue that the truth of these statements guarantees the truth of P_{n+1} . In a formal inductive argument, we show all three parts explicitly.

EXAMPLE 1.5

A binary tree is a tree in which no parent can have more than two children. Prove that a binary tree of height n has at most 2^n leaves.

Proof: If we denote the maximum number of leaves of a binary tree of height n by $l(n)$, then we want to show that $l(n) \leq 2^n$.

Basis: Clearly $l(0) = 1 = 2^0$ since a tree of height 0 can have no nodes other than the root, that is, it has at most one leaf.

Inductive Assumption:

$$l(i) \leq 2^i, \text{ for } i = 0, 1, \dots, n. \quad (1.4)$$

Inductive Step: To get a binary tree of height $n + 1$ from one of height n , we can create, at most, two leaves in place of each previous one. Therefore,

$$l(n + 1) = 2l(n).$$

Now, using the inductive assumption, we get

$$l(n + 1) \leq 2 \times 2^n = 2^{n+1}.$$

Thus, if our claim is true for n , it must also be true for $n + 1$. Since n can be any number, the statement must be true for all n . ■

Here we introduce the symbol ■ that is used in this book to denote the end of a proof.

Inductive reasoning can be difficult to grasp. It helps to notice the close connection between induction and recursion in programming. For example, the recursive definition of a function $f(n)$, where n is any positive integer, often has two parts. One involves the definition of $f(n + 1)$ in terms of $f(n), f(n - 1), \dots, f(1)$. This corresponds to the inductive step. The second part is the “escape” from the recursion, which is accomplished by defining $f(1), f(2), \dots, f(k)$ nonrecursively. This corresponds to the basis of induction. As in induction, recursion allows us to draw conclusions about all instances of the problem, given only a few starting values and using the recursive nature of the problem.

Sometimes, a problem looks difficult until we look at it in just the right way. Often looking at it recursively simplifies matters greatly.

EXAMPLE 1.6

A set $l_1, l_2, \dots, l_n$ of mutually intersecting straight lines divides the plane into a number of separated regions. A single line divides the plane into two parts, two lines generate four regions, three lines make seven regions, and so on. This is easily checked visually for up to three

lines, but as the number of lines increases it becomes difficult to spot a pattern. Let us try to solve this problem recursively.

Look at Figure 1.3 to see what happens if we add a new line, l_{n+1} , to existing n lines. The region to the left of l_1 is divided into two new regions, so is the region to the left of l_2 , and so on until we get to the last line. At the last line, the region to the right of l_n is also divided. Each of the n intersections then generates one new region, with one extra at the end. So, if we let $A(n)$ denote the number of regions generated by n lines, we see that

$$A(n+1) = A(n) + n + 1, n = 1, 2, \dots,$$

with $A(1) = 2$. From this simple recursion we then calculate $A(2) = 4$, $A(3) = 7$, $A(4) = 11$, and so on.

To get a formula for $A(n)$ and to show that it is correct, we use induction. If we conjecture that

$$A(n) = \frac{n(n+1)}{2} + 1,$$

then

$$\begin{aligned} A(n+1) &= \frac{n(n+1)}{2} + 1 + n + 1 \\ &= \frac{(n+1)(n+2)}{2} + 1 \end{aligned}$$

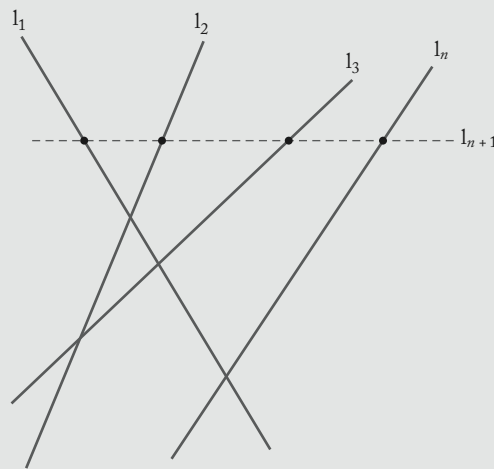


FIGURE 1.3

justifies the inductive step. The basis is easily checked, completing the argument.

In this example we have been a little less formal in identifying the basis, inductive assumption, and inductive step, but they are there and are essential. To keep our subsequent discussions from becoming too formal, we will generally prefer the style of this second example. However, if you have difficulty in following or constructing a proof, go back to the more explicit form of Example 1.5.

Proof by contradiction is another powerful technique that often works when everything else fails. Suppose we want to prove that some statement P is true. We then assume, for the moment, that P is false and see where that assumption leads us. If we arrive at a conclusion that we know is incorrect, we can lay the blame on the starting assumption and conclude that P must be true. The following is a classic and elegant example.

EXAMPLE 1.7

A rational number is a number that can be expressed as the ratio of two integers n and m so that n and m have no common factor. A real number that is not rational is said to be irrational. Show that $\sqrt{2}$ is irrational.

As in all proofs by contradiction, we assume the contrary of what we want to show. Here we assume that $\sqrt{2}$ is a rational number so that it can be written as

$$\sqrt{2} = \frac{n}{m}, \quad (1.5)$$

where n and m are integers without a common factor. Rearranging (1.5), we have

$$2m^2 = n^2.$$

Therefore, n^2 must be even. This implies that n is even, so that we can write $n = 2k$ or

$$2m^2 = 4k^2,$$

and

$$m^2 = 2k^2.$$

Therefore, m is even. But this contradicts our assumption that n and m have no common factors. Thus, m and n in (1.5) cannot exist and $\sqrt{2}$ is not a rational number.

This example exhibits the essence of a proof by contradiction. By making a certain assumption we are led to a contradiction of the assumption or some known fact. If all steps in our argument are logically sound, we must conclude that our initial assumption was false.

EXERCISES

1. With $S_1 = \{2, 3, 5, 7\}$, $S_2 = \{2, 4, 5, 8, 9\}$, and $U = \{1 : 10\}$, compute $\overline{S_1} \cup S_2$.
2. With $S_1 = \{2, 3, 5, 7\}$ and $S_2 = \{2, 4, 5, 8, 9\}$, compute $S_1 \times S_2$ and $S_2 \times S_1$.
3. For $S = \{2, 5, 6, 8\}$ and $T = \{2, 4, 6, 8\}$, compute $|S \cap T| + |S \cup T|$.
4. What relation between two sets S and T must hold so that $|S \cup T| = |S| + |T|$?
5. Show that for all sets S and T , $S - T = S \cap \overline{T}$.
6. Prove DeMorgan's laws, Equations (1.2) and (1.3), by showing that if an element x is in the set on one side of the equality, then it must also be in the set on the other side of the equality.
7. Show that if $S_1 \subseteq S_2$, then $\overline{S_2} \subseteq \overline{S_1}$.
8. Show that $S_1 = S_2$ if and only if $S_1 \cup S_2 = S_1 \cap S_2$.
9. Use induction on the size of S to show that if S is a finite set, then $|2^S| = 2^{|S|}$.
10. Show that if S_1 and S_2 are finite sets with $|S_1| = n$ and $|S_2| = m$, then

$$|S_1 \cup S_2| \leq n + m.$$

11. If S_1 and S_2 are finite sets, show that $|S_1 \times S_2| = |S_1| |S_2|$.
12. Consider the relation between two sets defined by $S_1 \equiv S_2$ if and only if $|S_1| = |S_2|$. Show that this is an equivalence relation.
13. Occasionally, we need to use the union and intersection symbols in a manner analogous to the summation sign Σ . We define

$$\bigcup_{p \in \{i, j, k, \dots\}} S_p = S_i \cup S_j \cup S_k \dots$$

with an analogous notation for the intersection of several sets.

With this notation, the general DeMorgan's laws are written as

$$\overline{\bigcup_{p \in P} S_p} = \bigcap_{p \in P} \overline{S_p}$$

and

$$\overline{\bigcap_{p \in P} S_p} = \bigcup_{p \in P} \overline{S_p}.$$

Prove these identities when P is a finite set.

14. Show that

$$S_1 \cup S_2 = \overline{\overline{S_1} \cap \overline{S_2}}.$$

15. Show that $S_1 = S_2$ if and only if

$$(S_1 \cap \overline{S_2}) \cup (\overline{S_1} \cap S_2) = \emptyset.$$

16. Show that

$$S_1 \cup S_2 - (S_1 \cap \overline{S_2}) = S_2.$$

17. Show that the distributive law

$$S_1 \cap (S_2 \cup S_3) = (S_1 \cap S_2) \cup (S_1 \cap S_3)$$

holds for sets.

18. Show that

$$S_1 \times (S_2 \cup S_3) = (S_1 \times S_2) \cup (S_1 \times S_3).$$

19. Give conditions on S_1 and S_2 necessary and sufficient to ensure that

$$S_1 = (S_1 \cup S_2) - S_2.$$

20. Use the equivalence defined in Example 1.4 to partition the set $\{2, 4, 5, 6, 9, 22, 24, 25, 31, 37\}$ into equivalence classes.

21. Show that if $f(n) = O(g(n))$ and $g(n) = O(f(n))$, then $f(n) = \Theta(g(n))$.

22. Show that $2^n = O(3^n)$, but $2^n \neq \Theta(3^n)$.

23. Show that the following order-of-magnitude results hold.

(a) $n^2 + 5 \log n = O(n^2)$.

(b) $3^n = O(n!)$.

(c) $n! = O(n^n)$.

24. Show that $(n^3 - 2n)/(n + 1) = \Theta(n^2)$.

25. Show that $\frac{n^3}{\log(n + 1)} = O(n^3)$ but not $O(n^2)$.

26. What is wrong with the following argument? $x = O(n^4)$, $y = O(n^2)$, therefore $x/y = O(n^2)$.

27. What is wrong with the following argument? $x = \Theta(n^4)$, $y = \Theta(n^2)$, therefore $x/y = \Theta(n^2)$.

28. Prove that if $f(n) = O(g(n))$ and $g(n) = O(h(n))$, then $f(n) = O(h(n))$.

29. Show that if $f(n) = O(n^2)$ and $g(n) = O(n^3)$, then

$$f(n) + g(n) = O(n^3)$$

and

$$f(n)g(n) = O(n^5).$$

In this case, is it true that $g(n)/f(n) = O(n)$?

30. Assume that $f(n) = 2n^2 + n$ and $g(n) = O(n^2)$. What is wrong with the following argument?

$$f(n) = O(n^2) + O(n),$$

so that

$$f(n) - g(n) = O(n^2) + O(n) - O(n^2).$$

Therefore,

$$f(n) - g(n) = O(n).$$

31. Show that if $f(n) = \Theta(\log_2 n)$, then $f(n) = \Theta(\log_{10} n)$.
32. Draw a picture of the graph with vertices $\{v_1, v_2, v_3\}$ and edges $\{(v_1, v_1), (v_1, v_2), (v_2, v_3), (v_2, v_1), (v_3, v_1)\}$. Enumerate all cycles with base v_1 .
33. Construct a graph with five vertices, ten edges, and no cycles.
34. Let $G = (V, E)$ be any graph. Prove the following claim: If there is any walk between $v_i \in V$ and $v_j \in V$, then there must be a simple path of length no larger than $|V| - 1$ between these two vertices.
35. Consider graphs in which there is at most one edge between any two vertices. Show that under this condition a graph with n vertices has at most n^2 edges.

36. Show that

$$\sum_{i=0}^n 2^i = 2^{n+1} - 1.$$

37. Show that

$$\sum_{i=1}^n \frac{1}{i^2} \leq 2 - \frac{1}{n}.$$

38. Prove that for all $n \geq 4$ the inequality $2^n < n!$ holds.

39. The *Fibonacci sequence* is defined recursively by

$$f(n+2) = f(n+1) + f(n), n = 1, 2, \dots,$$

with $f(1) = 1, f(2) = 1$. Show that

- (a) $f(n) = O(2^n)$,
 (b) $f(n) = \Omega(1.5^n)$.

40. Show that $\sqrt{8}$ is not a rational number.
41. Show that $2 - \sqrt{2}$ is irrational.
42. Show that $\sqrt{3}$ is irrational.
43. Prove or disprove the following statements.
 (a) The sum of a rational and an irrational number must be irrational.
 (b) The sum of two positive irrational numbers must be irrational.
 (c) The product of a nonzero rational and an irrational number must be irrational.
44. Show that every positive integer can be expressed as the product of prime numbers.
45. Prove that the set of all prime numbers is infinite.
46. A prime pair consists of two primes that differ by two. There are many prime pairs, for example, 11 and 13, 17 and 19. Prime triplets are three numbers $n \geq 2, n+2, n+4$ that are all prime. Show that the only prime triplet is $(3, 5, 7)$.
-

1.2 THREE BASIC CONCEPTS

Three fundamental ideas are the major themes of this book: **languages**, **grammars**, and **automata**. In the course of our study we will explore many results about these concepts and about their relationship to each other. First, we must understand the meaning of the terms.

Languages

We are all familiar with the notion of natural languages, such as English and French. Still, most of us would probably find it difficult to say exactly what the word “language” means. Dictionaries define the term informally as a system suitable for the expression of certain ideas, facts, or concepts, including a set of symbols and rules for their manipulation. While this gives us an intuitive idea of what a language is, it is not sufficient as a definition for the study of formal languages. We need a precise definition for the term.

We start with a finite, nonempty set Σ of symbols, called the **alphabet**. From the individual symbols we construct **strings**, which are finite sequences of symbols from the alphabet. For example, if the alphabet $\Sigma = \{a, b\}$, then *abab* and *aaabbba* are strings on Σ . With few exceptions, we will use lowercase letters *a, b, c, ...* for elements of Σ and *u, v, w, ...* for string names. We will write, for example,

$$w = abaaa$$

to indicate that the string named *w* has the specific value *abaaa*.

The **concatenation** of two strings *w* and *v* is the string obtained by appending the symbols of *v* to the right end of *w*, that is, if

$$w = a_1a_2 \cdots a_n$$

and

$$v = b_1b_2 \cdots b_m,$$

then the concatenation of *w* and *v*, denoted by *wv*, is

$$wv = a_1a_2 \cdots a_nb_1b_2 \cdots b_m.$$

The **reverse** of a string is obtained by writing the symbols in reverse order; if *w* is a string as shown above, then its reverse w^R is

$$w^R = a_n \cdots a_2a_1.$$

The **length** of a string *w*, denoted by $|w|$, is the number of symbols in the string. We will frequently need to refer to the **empty string**, which is a string with no symbols at all. It will be denoted by λ . The following simple relations

$$\begin{aligned} |\lambda| &= 0, \\ \lambda w &= w\lambda = w \end{aligned}$$

hold for all *w*.

Any string of consecutive symbols in some w is said to be a **substring** of w . If

$$w = vu,$$

then the substrings v and u are said to be a **prefix** and a **suffix** of w , respectively. For example, if $w = abbab$, then $\{\lambda, a, ab, abb, abba, abbab\}$ is the set of all prefixes of w , while bab, ab, b are some of its suffixes.

Simple properties of strings, such as their length, are very intuitive and probably need little elaboration. For example, if u and v are strings, then the length of their concatenation is the sum of the individual lengths, that is,

$$|uv| = |u| + |v|. \quad (1.6)$$

But although this relationship is obvious, it is useful to be able to make it precise and prove it. The techniques for doing so are important in more complicated situations.

EXAMPLE 1.8

Show that (1.6) holds for any u and v . To prove this, we first need a definition of the length of a string. We make such a definition in a recursive fashion by

$$\begin{aligned} |a| &= 1, \\ |wa| &= |w| + 1, \end{aligned}$$

for all $a \in \Sigma$ and w any string on Σ . This definition is a formal statement of our intuitive understanding of the length of a string: The length of a single symbol is one, and the length of any string is increased by one if we add another symbol to it. With this formal definition, we are ready to prove (1.6) by induction characters.

By definition, (1.6) holds for all u of any length and all v of length 1, so we have a basis. As an inductive assumption, we take that (1.6) holds for all u of any length and all v of length $1, 2, \dots, n$. Now take any v of length $n + 1$ and write it as $v = wa$. Then,

$$\begin{aligned} |v| &= |w| + 1, \\ |uv| &= |uwa| = |uw| + 1. \end{aligned}$$

By the inductive hypothesis (which is applicable since w is of length n),

$$|uw| = |u| + |w|$$

so that

$$|uv| = |u| + |w| + 1 = |u| + |v|.$$

Therefore, (1.6) holds for all u and all v of length up to $n + 1$, completing the inductive step and the argument.

If w is a string, then w^n stands for the string obtained by repeating w n times. As a special case, we define

$$w^0 = \lambda,$$

for all w .

If Σ is an alphabet, then we use Σ^* to denote the set of strings obtained by concatenating zero or more symbols from Σ . The set Σ^* always contains λ . To exclude the empty string, we define

$$\Sigma^+ = \Sigma^* - \{\lambda\}.$$

While Σ is finite by assumption, Σ^* and Σ^+ are always infinite since there is no limit on the length of the strings in these sets. A language is defined very generally as a subset of Σ^* . A string in a language L will be called a **sentence** of L . This definition is quite broad; any set of strings on an alphabet Σ can be considered a language. Later we will study methods by which specific languages can be defined and described; this will enable us to give some structure to this rather broad concept. For the moment, though, we will just look at a few specific examples.

EXAMPLE 1.9

Let $\Sigma = \{a, b\}$. Then

$$\Sigma^* = \{\lambda, a, b, aa, ab, ba, bb, aaa, aab, \dots\}.$$

The set

$$\{a, aa, aab\}$$

is a language on Σ . Because it has a finite number of sentences, we call it a finite language. The set

$$L = \{a^n b^n : n \geq 0\}$$

is also a language on Σ . The strings $aabb$ and $aaaabbbb$ are in the language L , but the string abb is not in L . This language is infinite. Most interesting languages are infinite.

Since languages are sets, the union, intersection, and difference of two languages are immediately defined. The complement of a language is defined with respect to Σ^* ; that is, the complement of L is

$$\overline{L} = \Sigma^* - L.$$

The reverse of a language is the set of all string reversals, that is,

$$L^R = \{w^R : w \in L\}.$$

The concatenation of two languages L_1 and L_2 is the set of all strings obtained by concatenating any element of L_1 with any element of L_2 ; specifically,

$$L_1 L_2 = \{xy : x \in L_1, y \in L_2\}.$$

We define L^n as L concatenated with itself n times, with the special cases

$$L^0 = \{\lambda\}$$

and

$$L^1 = L$$

for every language L .

Finally, we define the **star-closure** of a language as

$$L^* = L^0 \cup L^1 \cup L^2 \dots$$

and the **positive closure** as

$$L^+ = L^1 \cup L^2 \dots$$

EXAMPLE 1.10

If

$$L = \{a^n b^n : n \geq 0\},$$

then

$$L^2 = \{a^n b^n a^m b^m : n \geq 0, m \geq 0\}.$$

Note that n and m in the above are unrelated; the string $aabbbaabbb$ is in L^2 .

The reverse of L is easily described in set notation as

$$L^R = \{b^n a^n : n \geq 0\},$$

but it is considerably harder to describe $\overline{L}$ or L^* this way. A few tries will quickly convince you of the limitation of set notation for the specification of complicated languages.

Grammars

To study languages mathematically, we need a mechanism to describe them. Everyday language is imprecise and ambiguous, so informal descriptions in English are often inadequate. The set notation used in Examples 1.9 and 1.10 is more suitable, but limited. As we proceed we will learn about several language-definition mechanisms that are useful in different circumstances. Here we introduce a common and powerful one, the notion of a **grammar**.

A grammar for the English language tells us whether a particular sentence is well formed or not. A typical rule of English grammar is “a sentence can consist of a noun phrase followed by a predicate.” More concisely we write this as

$$\langle sentence \rangle \rightarrow \langle noun_phrase \rangle \langle predicate \rangle,$$

with the obvious interpretation. This is, of course, not enough to deal with actual sentences. We must now provide definitions for the newly introduced constructs $\langle noun_phrase \rangle$ and $\langle predicate \rangle$. If we do so by

$$\begin{aligned}\langle noun_phrase \rangle &\rightarrow \langle article \rangle \langle noun \rangle, \\ \langle predicate \rangle &\rightarrow \langle verb \rangle,\end{aligned}$$

and if we associate the actual words “a” and “the” with $\langle article \rangle$, “boy” and “dog” with $\langle noun \rangle$, and “runs” and “walks” with $\langle verb \rangle$, then the grammar tells us that the sentences “a boy runs” and “the dog walks” are properly formed. If we were to give a complete grammar, then in theory, every proper sentence could be explained this way.

This example illustrates the definition of a general concept in terms of simple ones. We start with the top-level concept, here $\langle sentence \rangle$, and successively reduce it to the irreducible building blocks of the language. The generalization of these ideas leads us to formal grammars.

DEFINITION 1.1

A grammar G is defined as a quadruple

$$G = (V, T, S, P),$$

where V is a finite set of objects called **variables**,
 T is a finite set of objects called **terminal symbols**,
 $S \in V$ is a special symbol called the **start** variable,
 P is a finite set of **productions**.

It will be assumed without further mention that the sets V and T are non-empty and disjoint.

The production rules are the heart of a grammar; they specify how the grammar transforms one string into another, and through this they define a language associated with the grammar. In our discussion we will assume that all production rules are of the form

$$x \rightarrow y,$$

where x is an element of $(V \cup T)^+$ and y is in $(V \cup T)^*$. The productions are applied in the following manner: Given a string w of the form

$$w = uxv,$$

we say the production $x \rightarrow y$ is applicable to this string, and we may use it to replace x with y , thereby obtaining a new string

$$z = uyv.$$

This is written as

$$w \Rightarrow z.$$

We say that w **derives** z or that z is derived from w . Successive strings are derived by applying the productions of the grammar in arbitrary order. A production can be used whenever it is applicable, and it can be applied as often as desired. If

$$w_1 \Rightarrow w_2 \Rightarrow \cdots \Rightarrow w_n,$$

we say that w_1 derives w_n and write

$$w_1 \xRightarrow{*} w_n.$$

The $*$ indicates that an unspecified number of steps (including zero) can be taken to derive w_n from w_1 .

By applying the production rules in a different order, a given grammar can normally generate many strings. The set of all such terminal strings is the language defined or generated by the grammar.

DEFINITION 1.2

Let $G = (V, T, S, P)$ be a grammar. Then the set

$$L(G) = \left\{ w \in T^* : S \xRightarrow{*} w \right\}$$

is the language generated by G .

If $w \in L(G)$, then the sequence

$$S \Rightarrow w_1 \Rightarrow w_2 \Rightarrow \cdots \Rightarrow w_n \Rightarrow w$$

is a **derivation** of the sentence w . The strings $S, w_1, w_2, \dots, w_n$, which contain variables as well as terminals, are called **sentential forms** of the derivation.

EXAMPLE 1.11

Consider the grammar

$$G = (\{S\}, \{a, b\}, S, P),$$

with P given by

$$\begin{aligned} S &\rightarrow aSb, \\ S &\rightarrow \lambda. \end{aligned}$$

Then

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aabb,$$

so we can write

$$S \xRightarrow{*} aabb.$$

The string $aabb$ is a sentence in the language generated by G , while $aaSbb$ is a sentential form.

A grammar G completely defines $L(G)$, but it may not be easy to get a very explicit description of the language from the grammar. Here, however, the answer is fairly clear. It is not hard to conjecture that

$$L(G) = \{a^n b^n : n \geq 0\},$$

and it is easy to prove it. If we notice that the rule $S \rightarrow aSb$ is recursive, a proof by induction readily suggests itself. We first show that all sentential forms must have the form

$$w_i = a^i S b^i. \tag{1.7}$$

Suppose that (1.7) holds for all sentential forms w_i of length $2i + 1$ or less. To get another sentential form (which is not a sentence), we can only apply the production $S \rightarrow aSb$. This gets us

$$a^i S b^i \Rightarrow a^{i+1} S b^{i+1},$$

so that every sentential form of length $2i + 3$ is also of the form (1.7). Since (1.7) is obviously true for $i = 1$, it holds by induction for all i . Finally, to get a sentence, we must apply the production $S \rightarrow \lambda$, and we see that

$$S \xRightarrow{*} a^n S b^n \Rightarrow a^n b^n$$

represents all possible derivations. Thus, G can derive only strings of the form $a^n b^n$.

We also have to show that all strings of this form can be derived. This is easy; we simply apply $S \rightarrow aSb$ as many times as needed, followed by $S \rightarrow \lambda$.

EXAMPLE 1.12

Find a grammar that generates

$$L = \{a^n b^{n+1} : n \geq 0\}.$$

The idea behind the previous example can be extended to this case. All we need to do is generate an extra b . This can be done with a production $S \rightarrow Ab$, with other productions chosen so that A can derive the language in the previous example. Reasoning in this fashion, we get the grammar $G = (\{S, A\}, \{a, b\}, S, P)$, with productions

$$\begin{aligned} S &\rightarrow Ab, \\ A &\rightarrow aAb, \\ A &\rightarrow \lambda. \end{aligned}$$

Derive a few specific sentences to convince yourself that this works.

The previous examples are fairly easy ones, so rigorous arguments may seem superfluous. But often it is not so easy to find a grammar for a language described in an informal way or to give an intuitive characterization of the language defined by a grammar. To show that a given language is indeed generated by a certain grammar G , we must be able to show (a) that every $w \in L$ can be derived from S using G and (b) that every string so derived is in L .

EXAMPLE 1.13

Take $\Sigma = \{a, b\}$, and let $n_a(w)$ and $n_b(w)$ denote the number of a 's and b 's in the string w , respectively. Then the grammar G with productions

$$\begin{aligned} S &\rightarrow SS, \\ S &\rightarrow \lambda, \\ S &\rightarrow aSb, \\ S &\rightarrow bSa \end{aligned}$$

generates the language

$$L = \{w : n_a(w) = n_b(w)\}.$$

This claim is not so obvious, and we need to provide convincing arguments.

First, it is clear that every sentential form of G has an equal number of a 's and b 's, since the only productions that generate an a , namely $S \rightarrow aSb$ and $S \rightarrow bSa$, simultaneously generate a b . Therefore, every element of $L(G)$ is in L . It is a little harder to see that every string in L can be derived with G .

Let us begin by looking at the problem in outline, considering the various forms $w \in L$ can have. Suppose w starts with a and ends with b . Then it has the form

$$w = aw_1b,$$

where w_1 is also in L . We can think of this case as being derived starting with

$$S \Rightarrow aSb$$

if S does indeed derive any string in L . A similar argument can be made if w starts with b and ends with a . But this does not take care of all cases, since a string in L can begin and end with the same symbol. If we write down a string of this type, say $aabbba$, we see that it can be considered as the concatenation of two shorter strings $aabb$ and ba , both of which are in L . Is this true in general? To show that this is indeed so, we can use the following argument: Suppose that, starting at the left end of the string, we count $+1$ for an a and -1 for a b . If a string w starts and ends with a , then the count will be $+1$ after the leftmost symbol and -1 immediately before the rightmost one. Therefore, the count has to go through zero somewhere in the middle of the string, indicating that such a string must have the form

$$w = w_1w_2,$$

where both w_1 and w_2 are in L . This case can be taken care of by the production $S \rightarrow SS$.

Once we see the argument intuitively, we are ready to proceed more rigorously. Again we use induction. Assume that all $w \in L$ with $|w| \leq 2n$ can be derived with G . Take any $w \in L$ of length $2n + 2$. If $w = aw_1b$, then w_1 is in L , and $|w_1| = 2n$. Therefore, by assumption,

$$S \overset{*}{\Rightarrow} w_1.$$

Then

$$S \Rightarrow aSb \overset{*}{\Rightarrow} aw_1b = w$$

is possible, and w can be derived with G . Obviously, similar arguments can be made if $w = bw_1a$.

If w is not of this form, that is, if it starts and ends with the same symbol, then the counting argument tells us that it must have the form

$w = w_1w_2$, with w_1 and w_2 both in L and of length less than or equal to $2n$. Hence again we see that

$$S \Rightarrow SS \xRightarrow{*} w_1S \xRightarrow{*} w_1w_2 = w$$

is possible.

Since the inductive assumption is clearly satisfied for $n = 1$, we have a basis, and the claim is true for all n , completing our argument.

Normally, a given language has many grammars that generate it. Even though these grammars are different, they are equivalent in some sense. We say that two grammars G_1 and G_2 are **equivalent** if they generate the same language, that is, if

$$L(G_1) = L(G_2).$$

As we will see later, it is not always easy to see if two grammars are equivalent.

EXAMPLE 1.14

Consider the grammar $G_1 = (\{A, S\}, \{a, b\}, S, P_1)$, with P_1 consisting of the productions

$$\begin{aligned} S &\rightarrow aAb|\lambda, \\ A &\rightarrow aAb|\lambda. \end{aligned}$$

Here we introduce a convenient shorthand notation in which several production rules with the same left-hand sides are written on the same line, with alternative right-hand sides separated by $|$. In this notation $S \rightarrow aAb|\lambda$ stands for the two productions $S \rightarrow aAb$ and $S \rightarrow \lambda$.

This grammar is equivalent to the grammar G in Example 1.11. The equivalence is easy to prove by showing that

$$L(G_1) = \{a^n b^n : n \geq 0\}.$$

We leave this as an exercise.

Automata

An automaton is an abstract model of a digital computer. As such, every automaton includes some essential features. It has a mechanism for reading input. It will be assumed that the input is a string over a given alphabet,

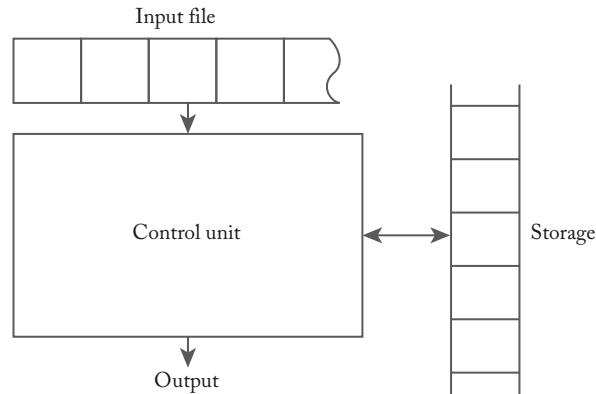


FIGURE 1.4

written on an **input file**, which the automaton can read but not change. The input file is divided into cells, each of which can hold one symbol. The input mechanism can read the input file from left to right, one symbol at a time. The input mechanism can also detect the end of the input string (by sensing an end-of-file condition). The automaton can produce output of some form. It may have a temporary **storage** device, consisting of an unlimited number of cells, each capable of holding a single symbol from an alphabet (not necessarily the same one as the input alphabet). The automaton can read and change the contents of the storage cells. Finally, the automaton has a **control unit**, which can be in any one of a finite number of **internal states**, and which can change state in some defined manner. Figure 1.4 shows a schematic representation of a general automaton.

An automaton is assumed to operate in a discrete time frame. At any given time, the control unit is in some internal state, and the input mechanism is scanning a particular symbol on the input file. The internal state of the control unit at the next time step is determined by the **next-state** or **transition function**. This transition function gives the next state in terms of the current state, the current input symbol, and the information currently in the temporary storage. During the transition from one time interval to the next, output may be produced or the information in the temporary storage changed. The term **configuration** will be used to refer to a particular state of the control unit, input file, and temporary storage. The transition of the automaton from one configuration to the next will be called a **move**.

This general model covers all the automata we will discuss in this book. A finite-state control will be common to all specific cases, but differences will arise from the way in which the output can be produced and the nature of the temporary storage. As we will see, the nature of the temporary storage governs the power of different types of automata.

For subsequent discussions, it will be necessary to distinguish between **deterministic automata** and **nondeterministic automata**.

A deterministic automaton is one in which each move is uniquely determined by the current configuration. If we know the internal state, the input, and the contents of the temporary storage, we can predict the future behavior of the automaton exactly. In a nondeterministic automaton, this is not so. At each point, a nondeterministic automaton may have several possible moves, so we can only predict a set of possible actions. The relation between deterministic and nondeterministic automata of various types will play a significant role in our study.

An automaton whose output response is limited to a simple “yes” or “no” is called an **accepter**. Presented with an input string, an accepter either accepts the string or rejects it. A more general automaton, capable of producing strings of symbols as output, is called a **transducer**.

EXERCISES

1. How many substrings aab are in ww^Rw , where $w = aabbab$?
2. Use induction on n to show that $|u^n| = n|u|$ for all strings u and all n .
3. The reverse of a string, introduced informally above, can be defined more precisely by the recursive rules

$$\begin{aligned} a^R &= a, \\ (wa)^R &= aw^R, \end{aligned}$$

for all $a \in \Sigma$, $w \in \Sigma^*$. Use this to prove that

$$(uv)^R = v^R u^R,$$

for all $u, v \in \Sigma^+$.

4. Prove that $(w^R)^R = w$ for all $w \in \Sigma^*$.
5. Let $L = \{ab, aa, baa\}$. Which of the following strings are in L^* : $abaabaaabaa$, $aaaabaaaa$, $baaaaabaaaab$, $baaaaabaa$? Which strings are in L^4 ?
6. Let $\Sigma = \{a, b\}$ and $L = \{aa, bb\}$. Use set notation to describe $\bar{L}$.
7. Let L be any language on a nonempty alphabet. Show that L and $\bar{L}$ cannot both be finite.
8. Are there languages for which $\bar{L}^* = (\bar{L})^*$?
9. Prove that

$$(L_1 L_2)^R = L_2^R L_1^R$$

for all languages L_1 and L_2 .

10. Show that $(L^*)^* = L^*$ for all languages.
11. Prove or disprove the following claims.
 - (a) $(L_1 \cup L_2)^R = L_1^R \cup L_2^R$ for all languages L_1 and L_2 .
 - (b) $(L^R)^* = (L^*)^R$ for all languages L .

12. Find a grammar for the language $L = \{a^n, \text{where } n \text{ is even}\}$.
 13. Find a grammar for the language $L = \{a^n, \text{where } n \text{ is even and } n > 3\}$.
 14. Find grammars for $\Sigma = \{a, b\}$ that generate the sets of

- (a) all strings with exactly two a 's.
- (b) all strings with at least two a 's.
- (c) all strings with no more than three a 's.
- (d) all strings with at least three a 's.
- (e) all strings that start with a and end with b .
- (f) all strings with an even number of b 's.

In each case, give convincing arguments that the grammar you give does indeed generate the indicated language.

15. Give a simple description of the language generated by the grammar with productions

$$\begin{aligned} S &\rightarrow aaA, \\ A &\rightarrow bS, \\ S &\rightarrow \lambda. \end{aligned}$$

16. What language does the grammar with these productions generate?

$$\begin{aligned} S &\rightarrow Aa, \\ A &\rightarrow B, \\ B &\rightarrow Aa. \end{aligned}$$

17. Let $\Sigma = \{a, b\}$. For each of the following languages, find a grammar that generates it.

- (a) $L_1 = \{a^n b^m : n \geq 0, m < n\}$.
- (b) $L_2 = \{a^{3n} b^{2n} : n \geq 2\}$.
- (c) $L_3 = \{a^{n+3} b^n : n \geq 2\}$.
- (d) $L_4 = \{a^n b^{n-2} : n \geq 3\}$.
- (e) $L_1 L_2$.
- (f) $L_1 \cup L_2$.
- (g) L_1^3 .
- (h) L_1^* .
- (i) $L_1 - \overline{L_4}$.

18. Find grammars for the following languages on $\Sigma = \{a\}$.

- (a) $L = \{w : |w| \bmod 3 > 0\}$.
- (b) $L = \{w : |w| \bmod 3 = 2\}$.
- (c) $w = \{|w| \bmod 5 = 0\}$.

19. Find a grammar that generates the language

$$L = \{ww^R : w \in \{a, b\}^+\}.$$

Give a complete justification for your answer.

20. Find three strings in the language generated by

$$S \rightarrow aSb|bSa|a.$$

21. Complete the arguments in Example 1.14, showing that $L(G_1)$ does in fact generate the given language.

22. Show that the grammar $G = (\{S\}, \{a, b\}, S, P)$, with productions

$$S \rightarrow SS | SSS | aSb | bSa | \lambda,$$

is equivalent to the grammar in Example 1.13.

23. Show that the grammars

$$S \rightarrow aSb|ab|\lambda$$

and

$$S \rightarrow aaSbb|aSb|ab|\lambda$$

are equivalent.

24. Show that the grammars

$$S \rightarrow aSb|bSa|SS|a$$

and

$$S \rightarrow aSb|bSa|a$$

are not equivalent.

1.3 SOME APPLICATIONS*

Although we stress the abstract and mathematical nature of formal languages and automata, it turns out that these concepts have widespread applications in computer science and are, in fact, a common theme that connects many specialty areas. In this section, we present some simple examples to give the reader some assurance that what we study here is not just a collection of abstractions, but is something that helps us understand many important, real problems.

Formal languages and grammars are used widely in connection with programming languages. In most of our programming, we work with a more or less intuitive understanding of the language in which we write. Occasionally though, when using an unfamiliar feature, we may need to refer to precise descriptions such as the syntax diagrams found in most

programming texts. If we write a compiler, or if we wish to reason about the correctness of a program, a precise description of the language is needed at almost every step. Among the ways in which programming languages can be defined precisely, grammars are perhaps the most widely used.

The grammars that describe a typical language like Pascal or C are very extensive. For an example, let us take a smaller language that is part of a larger one.

EXAMPLE 1.15

The rules for variable identifiers in C are

1. An identifier is a sequence of letters, digits, and underscores.
2. An identifier must start with a letter or an underscore.
3. Identifiers allow upper- and lower-case letters.

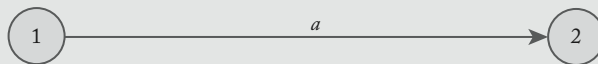
Formally, these rules can be described by a grammar.

$$\begin{aligned} \langle id \rangle &\rightarrow \langle letter \rangle \langle rest \rangle \mid \langle undrscr \rangle \langle rest \rangle \\ \langle rest \rangle &\rightarrow \langle letter \rangle \langle rest \rangle \mid \langle digit \rangle \langle rest \rangle \mid \langle undrscr \rangle \langle rest \rangle \mid \lambda \\ \langle letter \rangle &\rightarrow a \mid b \mid \dots \mid z \mid A \mid B \mid \dots \mid Z \\ \langle digit \rangle &\rightarrow 0 \mid 1 \mid \dots \mid 9 \\ \langle undrscr \rangle &\rightarrow - \end{aligned}$$

In this grammar, the variables are $\langle id \rangle$, $\langle letter \rangle$, $\langle digit \rangle$, $\langle undrscr \rangle$, and $\langle rest \rangle$. The letters, digits, and the underscore are terminals. A derivation of $a0$ is

$$\begin{aligned} \langle id \rangle &\Rightarrow \langle letter \rangle \langle rest \rangle \\ &\Rightarrow a \langle rest \rangle \\ &\Rightarrow a \langle digit \rangle \langle rest \rangle \\ &\Rightarrow a0 \langle rest \rangle \\ &\Rightarrow a0. \end{aligned}$$

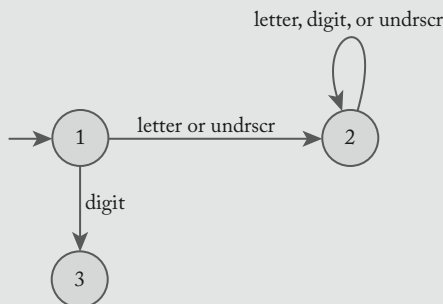
The definition of programming languages through grammars is common and very useful. But there are alternatives that are often convenient. For example, we can describe a language by an accepter, taking every string that is accepted as part of the language. To talk about this in a precise way, we will need to give a more formal definition of an automaton. We will do this shortly; for the moment, let us proceed in a more intuitive way.

**FIGURE 1.5**

An automaton can be represented by a graph in which the vertices give the internal states and the edges transitions. The labels on the edges show what happens (in terms of input and output) during the transition. For example, Figure 1.5 represents a transition from State 1 to State 2, which is taken when the input symbol is a . With this intuitive picture in mind, let us look at another way of describing C identifiers.

EXAMPLE 1.16

Figure 1.6 is an automaton that accepts all legal C identifiers. Some interpretation is necessary. We assume that initially the automaton is in State 1; we indicate this by drawing an arrow (not originating in any vertex) to this state. As always, the string to be examined is read left to right, one character at each step. When the first symbol is a letter or an underscore, the automaton goes into State 2, after which the rest of the string is immaterial. State 2 therefore represents the “yes” state of the accepter. Conversely, if the first symbol is a digit, the automaton will go into State 3, the “no” state, and remain there. In our solution, we assume that no input other than letters, digits, or underscores is possible.

**FIGURE 1.6**

Compilers and other translators that convert a program from one language to another make extensive use of the ideas touched on in these examples. Programming languages can be defined precisely through grammars,

as in Example 1.15, and both grammars and automata play a fundamental role in the decision processes by which a specific piece of code is accepted as satisfying the conditions of a programming language. The above example gives a first hint of how this is done; subsequent examples will expand on this observation.

Transducers will be discussed briefly in Appendix A; the following example previews this subject.

EXAMPLE 1.17

A binary adder is an integral part of any general-purpose computer. Such an adder takes two bit strings representing numbers and produces their sum as output. For simplicity, let us assume that we are dealing only with positive integers and that we use a representation in which

$$x = a_0a_1 \cdots a_n$$

stands for the integer

$$v(x) = \sum_{i=0}^n a_i 2^i.$$

This is the usual binary representation in reverse.

A serial adder processes two such numbers $x = a_0a_1 \cdots a_n$, and $y = b_0b_1 \cdots b_n$, bit by bit, starting at the left end. Each bit addition creates a digit for the sum as well as a carry digit for the next higher position. A binary addition table (Figure 1.7) summarizes the process.

A block diagram of the kind we saw when we first studied computers is given in Figure 1.8. It tells us that an adder is a box that accepts two bits and produces their sum bit and a possible carry. It describes what an adder does, but explains little about its internal workings. An automaton (now a transducer) can make this much more explicit.

		b_i	
		0	1
a_i	0	0 No carry	1 No carry
	1	1 No carry	0 Carry

FIGURE 1.7

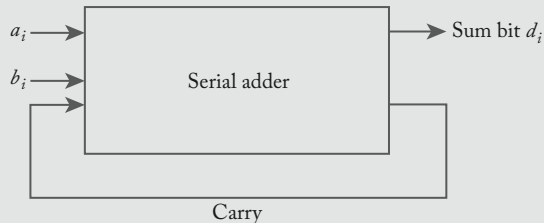


FIGURE 1.8

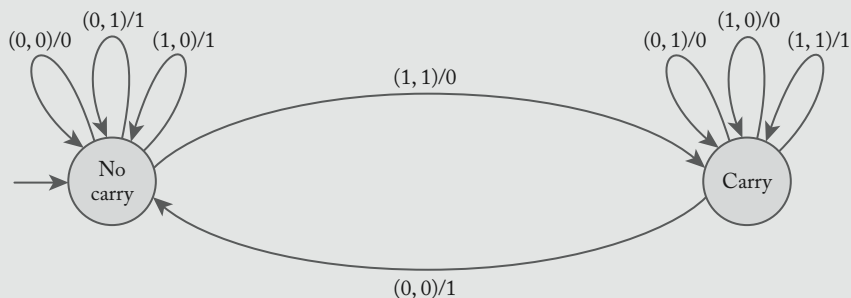


FIGURE 1.9

The input to the transducer are the bit pairs (a_i, b_i) ; the output will be the sum bit d_i . Again, we represent the automaton by a graph now labeling the edges $(a_i, b_i) / d_i$. The carry from one step to the next is remembered by the automaton via two internal states labeled “carry” and “no carry.” Initially, the transducer will be in state “no carry.” It will remain in this state until a bit pair $(1, 1)$ is encountered; this will generate a carry that takes the automaton into the “carry” state. The presence of a carry is then taken into account when the next bit pair is read. A complete picture of a serial adder is given in Figure 1.9. Follow this through with a few examples to convince yourself that it works correctly.

As this example indicates, the automaton serves as a bridge between the very high-level, functional description of a circuit and its logical implementation through transistors, gates, and flip-flops. The automaton clearly shows the decision logic, yet it is formal enough to lend itself to precise mathematical manipulation. For this reason, digital design methods rely heavily on concepts from automata theory.

EXERCISES

1. While passwords generally have few restrictions, they are normally not totally free. Suppose that in a certain system, passwords can be of arbitrary length but must contain at least one letter, a – z , and one number, 0–9. Construct a grammar that generates the set of such legal passwords.
2. Suppose that in some programming language, numbers are restricted as follows:
 - (a) A number may be signed or unsigned.
 - (b) The value field consists of two nonempty parts, separated by a decimal point.
 - (c) There is an optional exponent field. If present, this field must contain the letter e , followed by a signed two-digit integer.

Design a grammar set of such numbers.

3. Give a grammar for the set of integer numbers in C.
4. Design an accepter for integers in C.
5. Give a grammar that generates all real constants in C.
6. Suppose that a certain programming language permits only identifiers that begin with a letter, contain at least one but no more than three digits, and can have any number of letters. Give a grammar and an accepter for such a set of identifiers.
7. Modify the grammar in Example 1.15 so that the identifiers satisfy the following rules:
 - (a) C rules, except that an underscore cannot be the leftmost symbol.
 - (b) C rules, except that there can be at most one underscore.
 - (c) C rules, except that an underscore cannot be followed by a digit.
8. In the Roman number system, numbers are represented by strings on the alphabet $\{M, D, C, L, X, V, I\}$. Design an accepter that accepts such strings only if they are properly formed Roman numbers. For simplicity, replace the “subtraction” convention in which the number nine is represented by IX with an addition equivalent that uses $VIII$ instead.
9. We assumed that an automaton works in a framework of discrete time steps, but this aspect has little influence on our subsequent discussion. In digital design, however, the time element assumes considerable significance. In order to synchronize signals arriving from different parts of the computer, delay circuitry is needed. A unit-delay transducer is one that simply reproduces the input (viewed as a continual stream of symbols) one time unit later. Specifically, if the transducer reads as input a symbol a at time t , it will reproduce that symbol as output at time $t + 1$. At time $t = 0$, the transducer outputs nothing. We indicate this by saying that the transducer translates input $a_1a_2\cdots$ into output $\lambda a_1a_2\cdots$.

Draw a graph showing how such a unit-delay transducer might be designed for $\Sigma = \{a, b\}$.

10. An n -unit delay transducer is one that reproduces the input n time units later; that is, the input $a_1a_2\cdots$ is translated into $\lambda^n a_1a_2\cdots$, meaning again that the transducer produces no output for the first n time slots.
 - (a) Construct a two-unit delay transducer on $\Sigma = \{a, b\}$.
 - (b) Show that an n -unit delay transducer must have at least $|\Sigma|^n$ states.
11. The two's complement of a binary string, representing a positive integer, is formed by first complementing each bit, then adding one to the lowest-order bit. Design a transducer for translating bit strings into their two's complement, assuming that the binary number is represented as in Example 1.17, with lower-order bits at the left of the string.
12. Design a transducer to convert a binary string into octal. For example, the bit string 001101110 should produce the output 156.
13. Let $a_1a_2\cdots$ be an input bit string. Design a transducer that computes the parity of every substring of three bits. Specifically, the transducer should produce output

$$\begin{aligned}\pi_1 &= \pi_2 = 0, \\ \pi_i &= (a_{i-2} + a_{i-1} + a_i) \bmod 2, i = 3, 4, \dots\end{aligned}$$

For example, the input 110111 should produce 000001.

14. Design a transducer that accepts bit strings $a_1a_2a_3\cdots$ and computes the binary value of each set of three consecutive bits modulo five. More specifically, the transducer should produce $m_1, m_2, m_3, \dots$, where

$$\begin{aligned}m_1 &= m_2 = 0, \\ m_i &= (4a_i + 2a_{i-1} + a_{i-2}) \bmod 5, i = 3, 4, \dots\end{aligned}$$

15. Digital computers normally represent all information by bit strings, using some type of encoding. For example, character information can be encoded using the well-known ASCII system.

For this exercise, consider the two alphabets $\{a, b, c, d\}$ and $\{0, 1\}$, respectively, and an encoding from the first to the second, defined by $a \rightarrow 00$, $b \rightarrow 01$, $c \rightarrow 10$, $d \rightarrow 11$. Construct a transducer for decoding strings on $\{0, 1\}$ into the original message. For example, the input 010011 should generate as output *bad*.

16. Let x and y be two positive binary numbers. Design a transducer whose output is $\max(x, y)$.
-